Project: To-Do List API **CR ID:** CR-004 — Audit Trail

---

1. Technical Objective

Record a durable, chronological history of create/update/delete operations on `Task`, capturing old/new values, timestamp, and actor, without introducing Spring AOP and without assuming an authentication system that does not currently exist in this project.

2. Proposed Solution

Introduce a dedicated `AuditService`, called explicitly at the end of each relevant method in `TaskServiceImpl` (create, update, delete) within the same transaction as the operation itself. Each call persists one `TaskAudit` row capturing the operation type, old/new field values (as JSON), a timestamp, and an actor identity. A new read-only endpoint exposes a task's audit history.

An explicit, per-method service call is preferred over Spring AOP (per the CR's own direction) because it keeps the audit point visible directly in `TaskServiceImpl`'s code — easier to reason about for a project of this size, and avoids the indirection and "invisible" cross-cutting behavior that an `@Around` aspect introduces, which doesn't fit the project's existing "minimal implementation complexity" architecture rule.

3. Components to Modify

Component	File	Change
Task service	`service/impl/TaskServiceImpl.java`	Call `AuditService.record(...)` at the end of `create()`, `update()`, `delete()`, inside the existing transaction
Task controller	`controller/TaskController.java`	Add a new `GET /api/tasks/{id}/audit` endpoint

New components:

Component	Purpose
`entity/TaskAudit.java`	Persisted audit row
`repository/TaskAuditRepository.java`	`findByTaskIdOrderByTimestampAsc(Long taskId)`
`service/AuditService.java` (+ impl)	Builds and persists `TaskAudit` rows from before/after state
`dto/TaskAuditResponse.java`	Response shape for the audit-history endpoint
A system-identity constant/config (e.g. `app.audit.system-actor`)	Actor value used in the absence of authentication (see §9)

4. Database Changes (JPA/Hibernate)

```
@Entity
@Table(name = "TASK_AUDIT")
public class TaskAudit {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @Column(name = "TASK_ID", nullable = false)
    private Long taskId; // intentionally NOT a @ManyToOne — see note below

    @Enumerated(EnumType.STRING)
    @Column(name = "OPERATION", nullable = false, length = 20)
    private AuditOperation operation; // CREATE, UPDATE, DELETE
```

```
    @Lob
    @Column(name = "OLD_VALUES")
    private String oldValues; // JSON, null for CREATE

    @Lob
    @Column(name = "NEW_VALUES")
    private String newValues; // JSON, null for DELETE

    @Column(name = "ACTOR", nullable = false, length = 100)
    private String actor;

    @Column(name = "OCCURRED_AT", nullable = false)
    private Instant occurredAt;
}
```

- **`taskId` is a plain foreign-key column, not a JPA `@ManyToOne` relationship**, by design (FR-05): a `DELETE` audit entry must remain queryable after the referenced `Task` row no longer exists, which a non-nullable, cascading relationship mapping would complicate. No foreign key constraint with `ON DELETE CASCADE` is created for the same reason — a plain indexed column (`CREATE INDEX` on `TASK_ID`) is sufficient.
- Old/new values are stored as JSON text (`@Lob`) rather than individual columns per field, keeping the schema stable even as `Task` gains fields in the future (e.g. CR-001's `version`) without requiring a migration to the audit table itself.
- **Migration consideration:** new table, no existing data affected; `ddl-auto: update` is sufficient.

5. API Design

New endpoint:

```
GET /api/tasks/{id}/audit
```

Response `200 OK`:

```
[
  {
    "operation": "CREATE",
    "oldValues": null,
    "newValues": { "title": "Prepare report", "status": "TODO", "categoryId": 3 },
    "actor": "system",
    "occurredAt": "2026-10-10T09:00:00Z"
  },
  {
    "operation": "UPDATE",
    "oldValues": { "status": "TODO" },
    "newValues": { "status": "IN_PROGRESS" },
    "actor": "system",
    "occurredAt": "2026-10-12T14:30:00Z"
  }
]
```

`404 Not Found` if the task id has never existed (distinguish from "task was deleted but has audit history" — see §6 step 5 — which returns `200` with the history, since audit records intentionally outlive the task).

Existing `Task` endpoints' contracts are unchanged.

6. Processing Logic

1. **Create:** `TaskServiceImpl.create()` saves the new `Task` as today, then calls `auditService.record(taskId, CREATE, null, taskAsJson, actor)` before the transaction commits.
2. **Update:** `TaskServiceImpl.update()` captures the task's field values *before* applying the incoming changes (old state), applies and saves the changes as today, then calls `auditService.record(taskId, UPDATE, oldValuesJson, newValuesJson, actor)`.
3. **Delete:** `TaskServiceImpl.delete()` captures the task's full state *before* calling `taskRepository.delete(...)`, then calls `auditService.record(taskId, DELETE, oldValuesJson, null, actor)`.

4. All three calls happen inside the same `@Transactional` boundary as the underlying operation (FR-06) — if the save/delete throws, the audit insert is rolled back along with it, since both are part of one transaction.
5. **Retrieval:** `GET /api/tasks/{id}/audit` calls `taskAuditRepository.findByTaskIdOrderByOccurredAtAsc(id)` directly — it does **not** first check that the `Task` still exists, specifically so a deleted task's history remains retrievable (FR-05's intent).
6. **Actor identification (no auth system present):** every call uses a single, explicitly configured system-identity value (e.g. `"system"` or `"todo-api-service"`, from `app.audit.system-actor` in `application.yml`) rather than inventing a user/login concept. This is documented as a known limitation, not a hidden assumption — see §9.

7. Error Handling

Scenario	Behavior
Audit insert fails (e.g. transient DB error)	Propagates and rolls back the whole transaction, including the task create/update/delete (FR-06) — a task change without a corresponding audit entry is treated as a failure, not silently ignored
`GET /api/tasks/{id}/audit` for an id that never existed	`404 Not Found`
`GET /api/tasks/{id}/audit` for a deleted task that did exist	`200 OK` with its full history
`GET /api/tasks/{id}/audit` for a task with no history yet (should not normally occur, since CREATE always writes one entry)	`200 OK` with an empty list

8. Test Scenarios

See `TEST_CASES.md`. Summary: create/update/delete each produce exactly the right audit entry with correct old/new values; history is retrievable in order; history survives the task's deletion; a rolled-back operation produces no orphaned audit row; no endpoint allows writing to the audit table directly (FR-08 — verified by the absence of any such endpoint).

9. Technical Decision

Selected approach: Explicit `AuditService` calls from within `TaskServiceImpl`, inside the existing transaction, persisting JSON old/new-value snapshots to a dedicated `TASK_AUDIT` table.

Reason for selection: Matches the CR's own stated preference (explicit `AuditService` over AOP), keeps the audit point visible in the code that Tech Lead review will already be reading, and reuses the existing transactional/persistence infrastructure with no new dependency.

Alternatives considered:

- **Spring AOP (`@Around` advice on service methods):** rejected per explicit CR direction and because it would make the audit point implicit rather than visible at the call site — harder to reason about for a project this size.
- **Hibernate Envers (automatic entity-revision auditing):** rejected — pulls in a new dependency and a different, less transparent data model (revision tables per entity) than the simple, explicit `TASK_AUDIT` table this project needs; overkill relative to the actual requirement.
- **Database triggers:** rejected — moves business logic out of the application layer and out of version control, inconsistent with the project's architecture (all logic lives in Service/Repository, not in the database).

Open decision — actor identity: since no authentication exists in this project, "actor" is a fixed system-identity string, explicitly documented as a placeholder. If/when authentication is introduced to the project, this should be revisited to capture the real caller identity — this is flagged as an open question for the project owner, not resolved by this CR.

10. Technical Approval

- **Prepared By:** Backend Developer
- **Reviewed By:** Tech Lead
- **Approval Status:** Pending

- **Implementation Status:** Not Started